



Docker

Docker Swarm - Orchestrator steps in

Erfan Taheri

Orchestrator? TL;DR

- ▶ **Orchestrator schedules, deploys, scales, heals and manages workloads across nodes**
- ▶ **Automates desired-state, scaling, placement, health checks, and rolling updates**
- ▶ **Swarm: Docker's built-in orchestrator with tight engine integration and a simple experience**
- ▶ **Orchestrator = control plane + controllers that do these things for containers:**

Orchestrator? TL;DR

- ▶ **Orchestrator = control plane + controllers that do these things for containers:**
 - ▶ Maintain desired state (what you declared) vs actual state (what's running) and reconcile continuously.
 - ▶ Schedule containers to nodes (bin-packing, constraints, affinity).
 - ▶ Provide service discovery, load balancing, scaling, rolling updates, and self-healing.
 - ▶ Replaces manual docker run/VM ops with repeatable automation.
 - ▶ Enables resilient distributed apps (auto-restart, health checks, rollout/rollback).
 - ▶ Scales services horizontally and manages resources efficiently.
 - ▶ Standardizes networking, storage, and secrets across clusters.
 - ▶ faster deployments, lower MTTR, better resource utilization.

▶ Key Benefits of Swarm

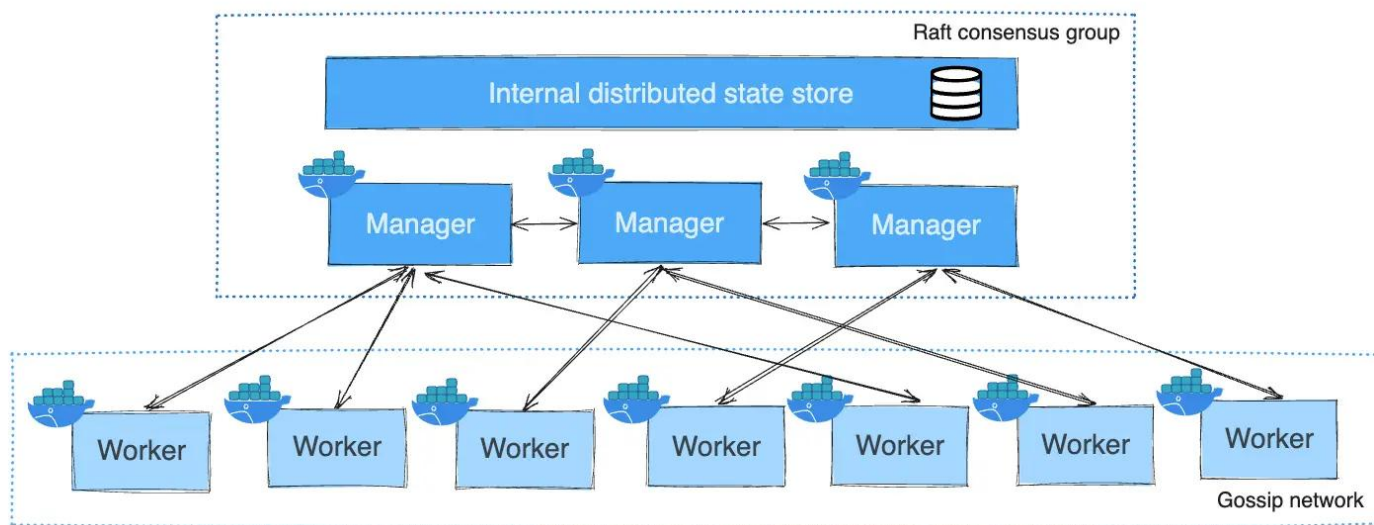
- ▶ **Simple & Fast:** Native to Docker, low learning curve, quick setup.
- ▶ **Lightweight:** Minimal resource overhead, ideal for small clusters & edge.
- ▶ **High Availability:** Built-in Raft consensus, leader election, auto-rescheduling.
- ▶ **Declarative Services:** Desired state management, rolling updates, rollbacks.
- ▶ **Integrated Networking & Security:** Routing mesh, overlay networks, TLS, secrets

▶ Use cases

- ▶ Small to medium clusters where simplicity is priority.
- ▶ Teams already using Docker CLI workflows.
- ▶ Development, testing, edge, or lab environments.

Docker Swarm architecture (overview)

- ▶ **Managers** (control plane) and **Workers** (run tasks).
- ▶ Running containers become **tasks** on workers
- ▶ Managers run the Raft cluster to store state and elect leader.
- ▶ Swarm managers use **Raft** for consensus (leader election, log replication, committed index).
- ▶ Swarm bundles scheduler, key-value store (Raft), and overlay networking tightly into Docker engine.



Why Raft consensus?

- ▶ A Swarm cluster may have multiple manager nodes.
- ▶ They must all agree on:
 - ▶ Services and tasks definitions
 - ▶ Node membership and status
 - ▶ Configs, secrets, networks
- ▶ Consensus ensures:
 - ▶ **Consistency** of cluster state
 - ▶ **Fault tolerance** when some managers fail
- ▶ without consensus, managers could diverge, leading to split-brain and data corruption.
- ▶ **Raft** = consensus algorithm for managing a replicated log.
- ▶ Used to ensure distributed systems agree on a single source of truth.
- ▶ Key properties:
 - ▶ Leader election
 - ▶ Log replication
 - ▶ Safety & fault tolerance

Raft in Docker Swarm

► Basics

- Manager nodes run a Raft group.
- One node is elected **Leader**; others are **Followers**.
- Cluster state is stored in the **Raft log** on disk.
- Updates (new services, scale changes, node join/leave) are appended to the log.
- Once committed, state is applied and replicated across all managers.
- All critical cluster data is persisted in Raft - no external database required.

Raft in Docker Swarm

► Raft group

- Manager nodes run a Raft group.
- One node is elected **Leader**; others are **Followers**.
- Cluster state is stored in the **Raft log** on disk.
- Updates (new services, scale changes, node join/leave) are appended to the log.
- Once committed, state is applied and replicated across all managers.
- All critical cluster data is persisted in Raft - no external database required.

Raft in Docker Swarm

► Log replication

- Leader appends new entries (e.g., service update) to its log.
- Sends entries to followers.
- Once a majority acknowledge, the entry is **committed**.
- All managers then apply the change to their local state.
- Replication ensures all managers converge to the same state

Raft in Docker Swarm

► Fault tolerance & cluster sizing

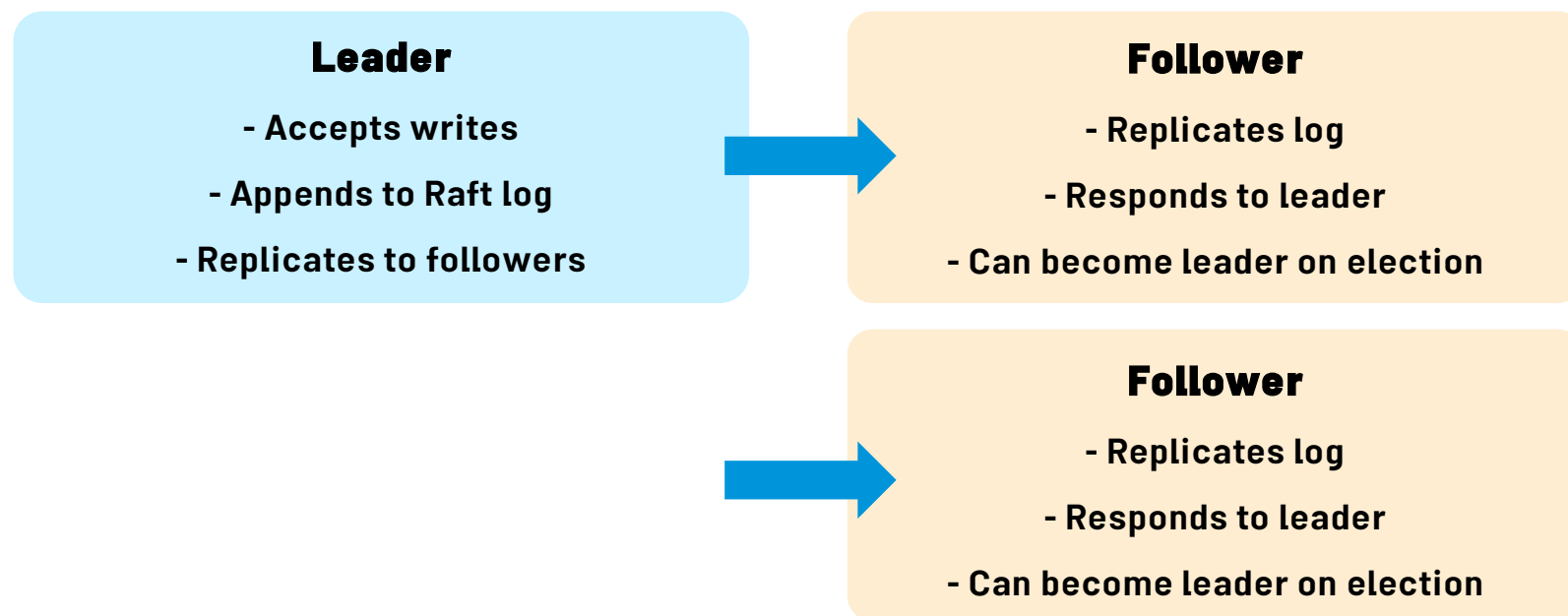
- Needs majority to operate.
- Recommended manager counts: 3 or 5, 7.
- Example:
 - 3 managers tolerate 1 failure.
 - 5 managers tolerate 2 failures.
- If quorum lost → cluster cannot accept changes (read-only mode).
- So, always run odd number of managers.
 - Raft tolerates up to $(N-1)/2$ failures and requires a majority or quorum of $(N/2)+1$ members to agree on values proposed to the cluster.

Raft in Docker Swarm

Data storage in Raft

- ▶ cluster state stored in **/var/lib/docker/swarm/raft/** on manager nodes.
- ▶ Includes:
 - ▶ Services, networks, configs, secrets, node info.
 - ▶ Regular snapshots are taken to compact logs.
 - ▶ Backup strategy: back up manager data folder.

Raft Consensus & State Storage



- Swarm stores cluster state (services, configs, secrets, membership) in the Raft replicated log on manager nodes. A majority (quorum) of managers must agree for writes to commit. Backup Raft data and plan manager topology.

Raft in Docker Swarm

► Sum up!

- Swarm managers rely on **Raft consensus** to agree on state.
- No external DB (like etcd) required.
- Provides consistency, fault tolerance, leader election.
- Requires quorum → run odd number of managers.
- Backup manager Raft data regularly.

Ports used by Docker Swarm

Port	Protocol	Used By	Purpose
2377	TCP	Managers & Workers	Cluster management communication (used for docker swarm join and control traffic)
7946	TCP / UDP	All nodes (managers & workers)	Node discovery & communication (gossip between nodes)
4789	UDP	All nodes (workers running tasks)	Overlay network traffic (VXLAN encapsulation for container-to-container networking across nodes)
50,000	TCP / UDP	Published services	Service ports (whatever ports you publish via docker service create -p)

Node Management - Commands

Operation	Command
List nodes	<code>docker node ls</code>
Inspect node	<code>docker node inspect <node></code>
Promote to manager	<code>docker node promote <node></code>
Demote to worker	<code>docker node demote <node></code>
Drain node (stop scheduling)	<code>docker node update --availability drain <node></code>
Set active	<code>docker node update --availability active <node></code>
Remove node	<code>docker node rm <node></code>
Join worker	<code>docker swarm join --token <worker-token> <manager-ip>:2377</code>
Join manager	<code>docker swarm join --token <manager-token> <manager-ip>:2377</code>

▶ A **Service** is the **definition of a containerized application** in a swarm.

▶ **Key points:**

▶ Declares the **desired state** (image, number of replicas, networks, ports).

▶ Swarm scheduler ensures **tasks (containers)** are running according to this state.

▶ Each task = 1 container instance managed by Swarm.

▶ **two modes:**

▶ **Replicated:** run N replicas across the cluster.

▶ **Global:** run one task on every node.

Why Use Services

▶ Simplifies deployment

- ▶ define the app once; swarm maintains replicas.

▶ High availability

- ▶ automatically reschedules failed tasks.

▶ Scaling made easy

- ▶ increase or decrease replicas with a single command.

▶ Rolling updates & rollbacks

- ▶ deploy new versions without downtime.

▶ Load balancing

- ▶ built-in service discovery and routing mesh.

Service Management - Commands

Action	Command / Notes
Create service	<code>docker service create --name web --replicas 3 -p 80:80 nginx</code>
List services	<code>docker service ls</code>
Inspect service	<code>docker service inspect <service></code>
Update service	<code>docker service update --image myimg:2.0 <service></code>
Scale service	<code>docker service scale <service>=5</code>
Remove service	<code>docker service rm <service></code>
Service tasks	<code>docker service ps <service></code>
Rolling update example	<code>docker service update --image myimg:2.0 --update-parallelism 2 --update-delay 10s <svc></code>

Services & Rolling Updates

- ▶ **Service: declarative unit (image, replicas, networks, update policy, constraints)**
 - ▶ A rolling update is a method to update services incrementally without downtime.
 - ▶ Swarm replaces old tasks (containers) step by step with new ones.
 - ▶ Ensures that the desired number of replicas is always running during update.
- ▶ **How Rolling Updates Work in Swarm**
 - ▶ Define the new image/version for the service.
 - ▶ Swarm schedules a batch of tasks to stop old containers and start new ones.
 - ▶ Waits for new tasks to become healthy before continuing to the next batch.
 - ▶ Continues until all replicas are updated.
- ▶ **Rolling update flags: --update-parallelism, --update-delay, --update-failure-action (pause/continue/rollback)**
 - ▶ Use HEALTHCHECK to drive graceful updates; prefer start-first when stateful readiness allows

Services & Rolling Updates

```
$ docker service update \  
  --image myapp:2.0 \  
  --update-parallelism 2 \  
  --update-delay 10s \  
  --update-failure-action rollback \  
  myapp
```

Services & Rolling Updates

► Benefits of Rolling Updates

- **Zero downtime:** service remains available during updates.
- **Control & safety:** can limit number of tasks updated simultaneously.
- **Easy rollback:** if update fails, revert to previous image/version.
- **Declarative updates:** desired state maintained automatically by Swarm.

► Best Practices

- Always test new images in staging before rolling updates.
- Use health checks to ensure task readiness before promoting traffic.
- Set sensible update-parallelism and update-delay for production clusters.
- Monitor logs and metrics during update.

Docker Swarm Routing Mesh (Service Mesh)

► What is Routing Mesh?

- **Routing Mesh** is Docker Swarm's built-in **load balancing mechanism**.
- It **routes external requests** to the correct service task across the cluster.
- Allows clients to access a service **on any node**, regardless of where the task is running.

► How Routing Mesh Works

- Swarm creates a **virtual IP (VIP)** for each service.
- Requests sent to any node on the published port are **intercepted by the node**.
- Node forwards the request to one of the **running service tasks** (containers), anywhere in the cluster.
- Load balancing is **automatic**, across all healthy tasks.

```
$ docker service create --name web --replicas 3 -p 80:80 nginx:alpine
```

Docker Swarm Routing Mesh (Service Mesh) - Sum up!

- ▶ Routing mesh exposes a service's published port on every node IP and routes to available tasks
- ▶ Ingress network + load-balancer component forwards traffic to tasks across nodes
- ▶ Load balancing is L4 by default; for advanced L7 routing use a service mesh or external proxy
- ▶ Consider health-based routing, session stickiness (external LB), and scaling the ingress layer for high throughput



Node A
IP:10.0.0.1
:80 Published



Node B
IP:10.0.0.2
:80 Published



Node C
IP:10.0.0.3
:80 Published

Tasks (containers) running across nodes Forwarded via Ingress network

Overlay Network

► What is an Overlay Network?

- **Overlay Network is a virtual network spanning multiple nodes.**
- **Connects containers/tasks across swarm nodes.**
- **Uses VXLAN to encapsulate traffic between nodes.**
- **Each service can attach to one or more overlay networks.**

► How It Works in Docker Swarm

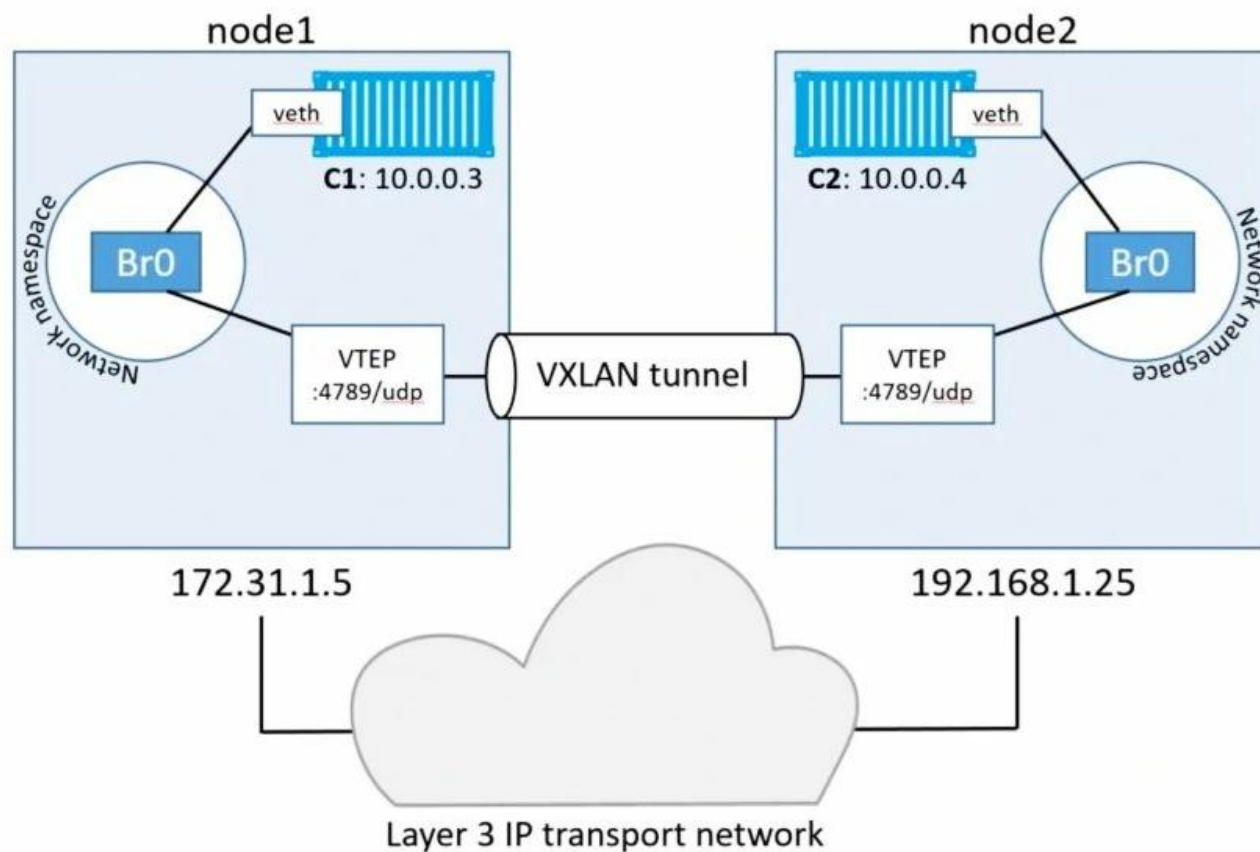
- **Swarm creates an overlay network.**
- **Each container gets a virtual IP within the overlay.**
- **Traffic is encapsulated via VXLAN and sent to the destination node.**
- **Swarm routing mesh delivers packets to the correct container.**

Overlay Network

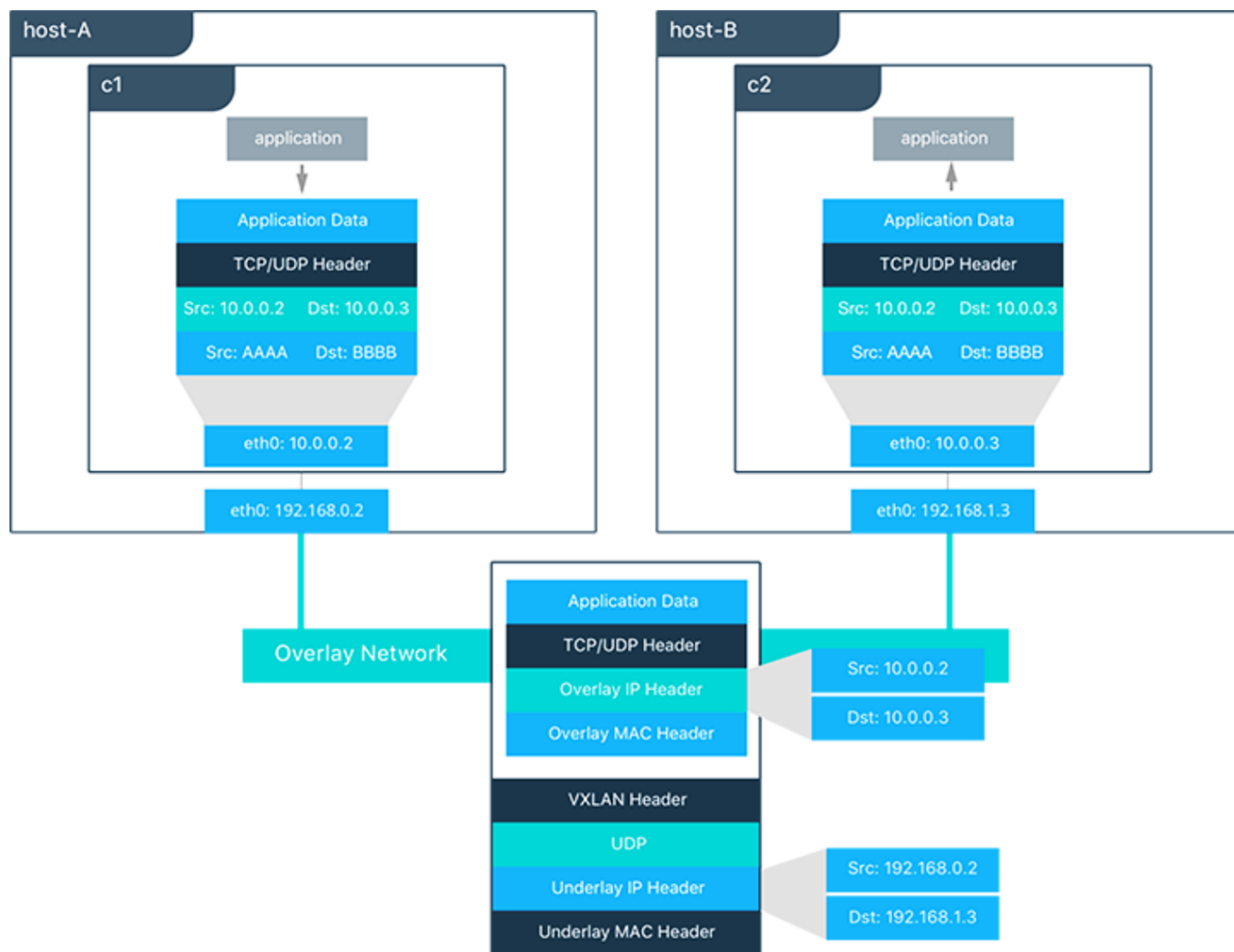
- ▶ **VXLAN (Virtual Extensible LAN)** is a network virtualization technology.
- ▶ Encapsulates **Layer 2 Ethernet frames inside Layer 3 UDP packets**.
- ▶ Enables **containers on different physical hosts** to communicate as if on the same LAN.
- ▶ Default technology for Docker Swarm **overlay networks**.

VXLAN allows Swarm to abstract network across nodes without changing the underlying physical network.

Overlay Network



Overlay Network



Overlay Network - sum up

- ▶ **VXLAN encapsulates Layer 2 frames inside UDP (tunnels) so overlay networks can span physical IP networks**
- ▶ **Docker overlay creates per-network VXLAN tunnels between nodes to forward container traffic across hosts**
- ▶ **Overlay control plane maintains mapping of endpoints to nodes; data plane uses VXLAN tunnels**
- ▶ **MTU/fragmentation: account for VXLAN/UDP overhead; reduce MTU on interfaces to avoid fragmentation**
- ▶ **Troubleshooting: check 'docker network inspect', ip link, and encapsulation with tcpdump/vxlan filters**

▶ Secrets

- ▶ **Secrets** are sensitive data (passwords, tokens, certificates) managed by Swarm.
- ▶ Stored **encrypted at rest** in the Raft log.
- ▶ Only sent to nodes running tasks that require them.
- ▶ Accessible inside containers as **files** in `/run/secrets/` mounted as tmpfs

▶ Configs

- ▶ **Configs** are non-sensitive configuration files for services.
- ▶ Shared across tasks and nodes as **read-only files**.
- ▶ Ideal for app configs, scripts, or any text-based configuration.
- ▶ Accessible inside containers at a specified **mount path**.

Secrets & Configs

Create a secret

```
$ docker secret create db_password ./password.txt
```

Create a config

```
$ docker config create app_config ./app_config.yml
```

Attach secret and config to a service

```
$ docker service create --name myapp \  
--secret db_password \  
--config app_config nginx:alpine
```

Secrets & Configs

- ▶ **Configs:** non-sensitive files stored in Raft, injected as files into containers (good for app config)
- ▶ **Secrets:** small blobs (passwords, keys) stored encrypted in Raft and delivered over mTLS to nodes that need them
- ▶ **Secrets** are mounted as tmpfs into the container with strict permissions; they are NOT baked into images
- ▶ **Enable autolock** to encrypt the manager key; unlock managers on restart using the key (protects Raft encryption keys)

Docker Stack - Deploying Compose on Swarm

▶ What is Docker Stack?

- ▶ **Docker Stack** is a higher-level abstraction to deploy a **group of interrelated services** in Swarm.
- ▶ Uses a **Compose file** (`docker-compose.yml`) to define multiple services, networks, and volumes.
- ▶ Swarm deploys all services in the stack with a single command.
- ▶ Each stack can be **isolated by name**, allowing multiple stacks on the same cluster.
- ▶ stacks make multi-service orchestration easier and reproducible.

▶ Why Use Docker Stack?

- ▶ **Simplifies multi-service deployment:** define app architecture in Compose.
- ▶ **Declarative management:** maintain desired state for all services in the stack.
- ▶ **Supports scaling:** scale services individually or as part of the stack.
- ▶ **Integrated networking:** automatically creates overlay networks for inter-service communication.

Docker Stack - Deploying Compose on Swarm

```
# Deploy a stack  
docker stack deploy -c docker-compose.yml  
mystack
```

```
# List all stacks  
docker stack ls
```

```
# View services in a stack  
docker stack services mystack
```

```
# Remove a stack  
docker stack rm mystack
```

Docker Stack - Deploying Compose on Swarm

Command	Description	Example
<code>docker stack deploy</code>	Deploy a new stack or update an existing one using a Compose file	<code>docker stack deploy -c docker-compose.yml mystack</code>
<code>docker stack ls</code>	List all deployed stacks	<code>docker stack ls</code>
<code>docker stack services <STACK></code>	List all services within a specific stack	<code>docker stack services mystack</code>
<code>docker stack ps <STACK></code>	List tasks/containers of a stack	<code>docker stack ps mystack</code>
<code>docker stack rm <STACK></code>	Remove a stack and all its associated services, networks, and volumes	<code>docker stack rm mystack</code>
<code>docker stack config <STACK></code>	View the resolved Compose file for a deployed stack	<code>docker stack config mystack</code>